```
export default class ConcertController extends Controller {
  static targets = ["tickets"]
  ticketsTarget: HTMLElement

  static values = { id: Number, soldOut: Boolean, ticketsRemaining: Number }
  soldOutValue: boolean
  ticketsRemainingValue: number

  soldOutValueChanged(): void {
    if (this.soldOutValue) {
      this.ticketsTarget.innerText = "Sold Out"
    } else {
      const ticketsRemaining = `${this.ticketsRemainingValue} Tickets Remaining`
      this.ticketsTarget.innerText = ticketsRemaining
    }
  }
}
```

It's really only a couple of lines of code after we're through declaring the targets and values, but the key is the name of the method: soldOutValueChanged. It's the method hook called by Stimulus when the soldOutValue changes, or when the associated data attribute, data-concert-sold-out-value, changes. And, if you look back at the SoldOutDataController, that's exactly what it does—change the value of that data attribute.

Again, you could argue that we're creating a dependency between the two controllers by hard-coding the name of the data attribute of one controller in another. I'm not very bothered by that since the naming pattern is consistent and enforced by Stimulus. You could make a separate data attribute containing the name of the real data attribute to break that dependency, but that seemed needlessly confusing here.

Anyway, this works. The SoldOutDataController calls the server, returns its JSON, and changes the correct data attributes. The ConcertController detects the change, the soldOutValueChanged method is invoked, and the text on the page is updated automatically.

You can prove this works by invoking the Rake task rails north_by:sell_out_shows, which is in the chapter_09/03 version of the code that sells out three shows at